

实验题目： vm_fault 内存映射

一、 实验内容

按实验内容编写驱动程序及测试程序

编译驱动程序及测试程序

在嵌入式设备上加载驱动程序并进行测试

写一个简单的字符设备驱动程序，要求：

在加载驱动程序时利用 `get_free_pages` 函数申请一片 64KB 的连续物理地址空间；

利用 `vma fault` 机制实现对申请到的 64KB 地址空间进行内存映射；

编写应用程序利用 `mmap` 进行内存映射，读写映射内存区域，通过打印输出观察具体每个页面实际进行内存映射的时机；

在卸载驱动程序时利用 `free_pages` 释放申请到的 64KB 空间。

注意事项：

在 `fault` 函数中 `vmf->virtual_address - vma->vm_start` 为虚拟地址 `vmf->virtual_address` 相对 `vma->vm_start` 的偏移，如果利用 `get_free_pages` 申请到的 64KB 地址空间的首地址为 `offset`，则用户态虚拟地址 `vmf->virtual_address` 对应的内核态逻辑地址即为 `virtaddr = vmf->virtual_address - vma->vm_start + offset`，利用 `virt_to_page(virtaddr)` 即可得到 `vma fault` 返回需要的 `struct page*` 指针。

在编写驱动程序及测试程序时，在关键位置增加打印信息，使得测试结果打印输出可以显示驱动程序加载卸载以及应用程序调用驱动程序的过程。

驱动程序打印调用 `printk` 函数

在 `ubuntu` 虚拟机上完成程序验证

二、 实验原理（代码以及注释）

demo_drv4.c:

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/types.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include <asm/system.h>
#include <linux/slab.h>
#include <linux/mm.h>
#include <linux/gfp.h>
#include <asm/page.h>
```

```
unsigned long ADDRESS;           //驱动程序申请的空间的地址
int times = 0;
```

```
#define DEMO_MAJOR 98
#define DEMO_MINOR 0
#define DEVICE_CNT 1
#define DEVICE_NAME "demo_drv4"
```

```
static int Demo_open(struct inode *inode, struct file *filp)
{
    printk("Demo_open[--kernel--]\n");
```

```

    return 0;}
static int Demo_release(struct inode *inode,struct file *filp)
{
    printk("Demo_release_one[--kernel--]\n");
    return 0;}
void simple_vma_open(struct vm_area_struct *vma) {
    printk(KERN_NOTICE "Simple VMA open, virt %lx, phys %lx\n", vma->vm_start,
vma->vm_pgoff << PAGE_SHIFT); }
void simple_vma_close(struct vm_area_struct *vma) {
    printk(KERN_NOTICE "Simple VMA close\n"); }

//缺页错，只在发生缺页错时才将虚拟地址映射到内存空间
int simple_vma_fault(struct vm_area_struct *vma, struct vm_fault *vmf) {
    struct page *pageptr;
    unsigned long baseaddr = ADDRESS; /* 驱动程序申请空间的首地址 */
    unsigned long virtaddr = vmf->virtual_address - vma->vm_start + baseaddr;//找到产生缺页错
的地址在内核态对应的地址（offset+baseaddr）

    pageptr = virt_to_page(virtaddr);//拿到 page 指针
    if (!pageptr)
        return VM_FAULT_SIGBUS;
    get_page(pageptr);
    /* got it, now increment the count */
    vmf->page = pageptr;
    printk("simple_vma_fault[--kernel--]: map Virtual Address %p\n", vmf-> virtual_address);
    return 0;
}

static struct vm_operations_struct simple_fault_vm_ops = {
    .open = simple_vma_open,
    .close = simple_vma_close,
    .fault = simple_vma_fault,};

static int simple_fault_mmap(struct file *filp, struct vm_area_struct *vma) {
    unsigned long offset = vma->vm_pgoff << PAGE_SHIFT;
    if (offset >= __pa(high_memory) || (filp->f_flags & O_SYNC))
        vma->vm_flags |= VM_IO;
    vma->vm_flags |= VM_RESERVED;
//对虚拟内存的操作赋值
    vma->vm_ops = &simple_fault_vm_ops;//表明实际上就是采用 vma_fault 机制来做内存映射，
产生缺页错才调用
    simple_vma_open(vma);

    printk("simple_fault_mmap[--kernel--]\n");
    return 0;}

struct file_operations Demo_ops ={

```

```

.open = Demo_open,
.release = Demo_release,
.mmap = simple_fault_mmap,});

static int __init Demo_Init(void)
{
    int ret = -1;
    ret = register_chrdev(DEMO_MAJOR,DEVICE_NAME,&Demo_ops);
    if(ret){
        printk("Demo_module failed with %d\n[--kernel--]\n",ret);
        return ret;
    }else{
        printk("Demo_driver register success!!![--kernel--]\n");
    }
}
//一共申请 64KB 内存，每页 4KB，一共 16 页，也就是 2^4，故参数为 4
ADDRESS = __get_free_pages(GFP_KERNEL,4);
if(ADDRESS == 0)
    printk("gg----kernel");
    return 0;
}
static void __exit Demo_Exit(void)
{
    unregister_chrdev(DEMO_MAJOR,DEVICE_NAME);
    free_pages(ADDRESS,4);
    printk("Demo_driver register success!!![--kernel--]\n");}

/* 模块元信息 */
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple driver module");

/* 注册初始化/退出函数 */
module_init(Demo_Init);
module_exit(Demo_Exit);

```

test_demo_drv1.c:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/mman.h>

#define DEVICE_NAME "/dev/demo_drv4" // 需与驱动中的 DEVICE_NAME 一致
#define PAGE_SIZE (4 * 1024) // 4KB 页大小
#define SHARE_MEM_SIZE (16*PAGE_SIZE) // 64KB 共享内存
#define SHARE_MEM_PAGE_COUNT 16 // 16 页

```

```

int main()
{
    int fd;           // 设备文件描述符
    int ret;          // 系统调用返回值
    int loop = 0;     // 循环计数器
    char *ptrdata = NULL;

    printf("\nStart Demo_driver test\n\n");

    fd = open(DEVICE_NAME, O_RDWR);
    // 打开设备文件
    if (fd == -1) {
        printf("open device %s failed\n", DEVICE_NAME);
    }
    else
    {
        ptrdata=(char*)mmap(0,          SHARE_MEM_SIZE,          PROT_READ|PROT_WRITE,
MAP_SHARED, fd, 0);
        printf("%p\n", ptrdata);
        for(loop=0;loop<SHARE_MEM_PAGE_COUNT;loop++)
        {
            printf("[%p-----]\n", ptrdata+4096*loop);//打印每一页内存的起始地址
        }
        for(loop=0;loop<SHARE_MEM_PAGE_COUNT;loop++)
        {
            printf("[%p: %d-----]\n", ptrdata+4096*loop, *(ptrdata+4096*loop));//打印每一页起
始地址处的字节值
        }
        for(loop=0;loop<SHARE_MEM_PAGE_COUNT;loop++)
        {
            printf("[%p: %d-----]\n", ptrdata+4096*loop, *(ptrdata+4096*loop+4));//打印每一页
偏移 4 字节位置处的内容
        }
        munmap(ptrdata,SHARE_MEM_SIZE);
        ret=close(fd); //若 ret 的值为 0，表示设备成功关闭
        printf("ret = %d\n",ret);
        printf("close Demo_drive test\n");
    }
    return 0;
}

```

三、实验结果及其分析

编译成功后，加载驱动程序，添加设备节点，对测试程序编译，运行测试程序：

```

root@ubuntu:linux# cd test4
root@ubuntu:test4# make clean
rm -f *.ko *.mod.c *.o -core.depend
root@ubuntu:test4# make
make -C /lib/modules/3.2.14/build M=/home/linux/test4 modules
make[1]: 正在进入目录 `/usr/src/linux-source-3.2.0'
  CC [M] /home/linux/test4/demo_drv4.o
/home/linux/test4/demo_drv4.c: 在函数 'simple_vma_fault' 中:
/home/linux/test4/demo_drv4.c:47: 警告: 初始化将指针赋给整数, 未作类型转换
Building modules, stage 2.
MODPOST 1 modules
  CC /home/linux/test4/demo_drv4.mod.o
  LD [M] /home/linux/test4/demo_drv4.ko
make[1]:正在离开目录 `/usr/src/linux-source-3.2.0'
root@ubuntu:test4# insmod demo_drv4.ko
root@ubuntu:test4# mknod /dev/demo_drv4 c 98 0
root@ubuntu:test4# gcc test_demo_drv4.c -o test
root@ubuntu:test4# ./test

```

打印输出三次的阅读信息:

打印每一页内存的起始地址

打印每一页起始地址处的字节值

打印每一页偏移 4 字节位置处的内容

```
Start Demo_driver test
```

```

0x7f364b125000
[0x7f364b125000-----]
[0x7f364b126000-----]
[0x7f364b127000-----]
[0x7f364b128000-----]
[0x7f364b129000-----]
[0x7f364b12a000-----]
[0x7f364b12b000-----]
[0x7f364b12c000-----]
[0x7f364b12d000-----]
[0x7f364b12e000-----]
[0x7f364b12f000-----]
[0x7f364b130000-----]
[0x7f364b131000-----]
[0x7f364b132000-----]
[0x7f364b133000-----]
[0x7f364b134000-----]
[0x7f364b125000: -92-----]
[0x7f364b126000: -92-----]
[0x7f364b127000: -92-----]
[0x7f364b128000: -92-----]
[0x7f364b129000: -92-----]
[0x7f364b12a000: -92-----]
[0x7f364b12b000: -92-----]
[0x7f364b12c000: -92-----]
[0x7f364b12d000: -92-----]
[0x7f364b12e000: -92-----]
[0x7f364b12f000: -92-----]
[0x7f364b130000: -92-----]

```

```

[0x7f364b131000: 0-----]
[0x7f364b132000: 97-----]
[0x7f364b133000: -92-----]
[0x7f364b134000: -92-----]
[0x7f364b125000: 67-----]
[0x7f364b126000: 80-----]
[0x7f364b127000: 81-----]
[0x7f364b128000: 89-----]
[0x7f364b129000: 87-----]
[0x7f364b12a000: -122-----]
[0x7f364b12b000: 79-----]
[0x7f364b12c000: 3-----]
[0x7f364b12d000: -93-----]
[0x7f364b12e000: 45-----]
[0x7f364b12f000: 61-----]
[0x7f364b130000: 59-----]
[0x7f364b131000: 0-----]
[0x7f364b132000: 112-----]
[0x7f364b133000: 53-----]
[0x7f364b134000: 63-----]
ret = 0
close Demo_drive test

```

最后打印日志的时候，发现即使 16 页都被读了三次，但是每一页的内存映射只做了一次——只在产生缺页错时做内存映射（一共 16 次）

```
[ 9889.156745] Demo_open[--kernel--]
[ 9889.156753] Simple VMA open, virt 7f364b125000, phys 0
[ 9889.156755] simple_fault_mmap[--kernel--]
[ 9889.156808] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12500
0
[ 9889.156925] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12600
0
[ 9889.156963] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12700
0
[ 9889.157000] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12800
0
[ 9889.157037] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12900
0
[ 9889.157074] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12a00
0
[ 9889.157112] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12b00
0
[ 9889.157148] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12c00
0
[ 9889.157186] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12d00
0
[ 9889.157223] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12e00
0
[ 9889.157260] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b12f00
0
[ 9889.157298] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b13000
0
[ 9889.157321] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b13100
0
[ 9889.157344] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b13200
0
[ 9889.157368] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b13300
0
[ 9889.157392] simple_vma_fault[--kernel--]: map Virtual Address 00007f364b13400
0
[ 9889.157444] Simple VMA close
[ 9889.157447] Demo_release_one[--kernel--]
```